

Sort Colors: The Dutch National Flag (DNF) Algorithm

A Single-Pass Sorting Strategy for Three Distinct Values

1. The Problem Statement

- **Goal:** Sort an array containing n objects colored Red, White, or Blue.
- **Constraints:**
 - Solve **in-place** (Space: $O(1)$).
 - Single pass (**Time: $O(n)$**).
 - Represented by integers: 0 (Red), 1 (White), 2 (Blue).
 - **Rule:** Do not use built-in library sort functions.

2. The Algorithm: Dutch National Flag (DNF)

Invented by Edsger W. Dijkstra, this algorithm uses **three pointers** to partition the array into four distinct zones during execution.

The Four Zones:

1. [$\text{low} \dots \text{low}-1$]: **0s** (Red)
2. [$\text{low} \dots \text{mid}-1$]: **1s** (White)
3. [$\text{mid} \dots \text{high}$]: **Unknown** (To be processed)
4. [$\text{high}+1 \dots \text{end}$]: **2s** (Blue)

3. The Pointers

- **low:** Boundary for 0s. Everything before this is guaranteed to be 0.
- **mid:** The current element pointer. Everything between low and mid is guaranteed to be 1.
- **high:** Boundary for 2s. Everything after this is guaranteed to be 2.

4. Implementation Logic

We iterate while $\text{mid} \leq \text{high}$:

If <code>nums[mid]</code> is...	Action	Why?
0	<code>swap(mid, low), low++, mid++</code>	Move 0 to the front zone.
1	<code>mid++</code>	Keep 1 in the middle zone.
2	<code>swap(mid, high), high--</code>	Move 2 to the end zone.

5. Visual Walkthrough

Input: [2, 0, 2, 1, 1, 0]

Phase 1: Handling 2s

[2, 0, 2, 1, 1, 0] → mid is 2, swap with high. [0, 0, 2, 1, 1, 2] → high decrements.

Phase 2: Handling 0s

[0, 0, 2, 1, 1, 2] → mid is 0, swap with low. [0, 0, 2, 1, 1, 2] → Both low and mid increment.

Phase 3: The Result

[0, 0, 1, 1, 2, 2] - **0s** are pushed to the start. - **2s** are pushed to the end. - **1s** naturally settle in the middle.

6. Code Overview

```
while (mid <= high) {
    if (nums[mid] == 0) {
        swapValues(nums, mid, low);
        low++; mid++;
    } else if (nums[mid] == 1) {
        mid++;
    } else {
        swapValues(nums, mid, high);
        high--;
    }
}
```

7. Complexity Analysis

- **Time Complexity:** $O(N)$
 - Each element is visited at most once.
 - **Space Complexity:** $O(1)$
 - Sorting is done in-place with no auxiliary data structures.
-

8. Summary for the Team

- **Efficiency:** Faster than general-purpose sorts ($O(N \log N)$) because we leverage the fact that there are only 3 unique values.
- **Stability:** This specific DNF implementation is not stable, but it is optimal for space and time.
- **Application:** Useful in partitioning tasks (e.g., QuickSort's 3-way partition).